The Superior University, Lahore

LAB-TASK (6) (Spring-2026)

Course Title:	Programming for AI (Lab)				Course Code:	-	Credit Hours	1
Instructor:	Sir Rasikh				Program:	BSDS		
Semester:	4th	Batch:	Morning	Section:	4A	Date:	-	
Time Allowed:	-				Maximum Marks:		N/A	
Student's Name:	Shumaila Maryam				Roll No.	SU92-BSDSM-S25-062		

<u>Work Assigned:</u>

Question # 01:

Task 1; Animal Herd Detection (with Map Alert for Bonus Marks) • Detects groups of animals in an image/video.

• Uses OpenCV + YOLO/HAAR Cascade for object detection.

• Optionally, triggers alerts on a map (e.g., marking herd location).

• Must have a front-end based on flask Bonus Feature: Use Any Free API (or OpenStreetMap) to mark the detected location.

DEPARTMENT OF SOFTWARE ENGINEERING

Animal Herd Detection System

Introduction

The Animal Herd Detection System is a comprehensive deep learning solution designed to detect and classify animal species in real-time from images and videos. This project demonstrates advanced computer vision techniques, object detection algorithms, web application development, and geospatial integration. Built using Python with Flask, OpenCV, and YOLOv3, the system implements a state-of-the-art object detection model capable of identifying 10 different animal species with high accuracy and confidence.

The system leverages the pre-trained YOLOv3 (You Only Look Once version 3) model trained on the COCO (Common Objects in Context) dataset, which contains 80 object classes including multiple animal species. The web-based interface allows users to upload images or videos (up to 500MB), specify GPS coordinates for detection location, and receive comprehensive detection results including bounding boxes, species classification, confidence scores, and interactive geospatial maps showing herd locations.

Why I Created This

The primary objective of this project is to demonstrate and strengthen comprehensive deep learning and computer vision skills across the entire AI development pipeline. Through this implementation, I aimed to:

- **Computer Vision Mastery:** Understand and implement state-of-the-art object detection algorithms (YOLOv3) and learn how convolutional neural networks process visual information.
- **Deep Learning Deployment:** Gain expertise in deploying pre-trained models using OpenCV's DNN module for efficient real-time inference.
- **Multimedia Processing:** Develop skills in processing both images and videos frame-by-frame, handling different codecs, and generating annotated output media.
- **Web Application Development:** Create production-ready web interfaces using Flask framework, handle file uploads, manage sessions, and provide responsive user experiences.
- **Geospatial Integration:** Implement interactive mapping with Folium library to visualize detection locations and herd distributions on real-world geography.
- **Real-time Processing:** Optimize inference pipelines for real-time performance, implementing techniques like Non-Maximum Suppression (NMS) to eliminate redundant detections.
- **Full-Stack AI Development:** Experience the complete workflow from model selection, integration, web backend development, frontend design, to deployment considerations.

- **Practical Problem Solving:** Address real-world challenges in wildlife monitoring, agricultural livestock management, and conservation efforts through AI-powered solutions.

This project serves as both an educational exercise and a practical demonstration of building production-ready computer vision systems capable of solving real-world problems in agriculture, wildlife conservation, and animal management.

How It Works

1. System Initialization and Model Loading

The system begins by initializing the YOLOv3 model using OpenCV's DNN module. The model consists of:

- YOLOv3 weights file (237MB) - Pre-trained network parameters
- Network configuration (yolov3.cfg) - Architecture definition with 106 layers
- COCO class names - 80 object categories including 10 animal species
- Output layer configuration - Three detection scales for multi-scale object detection

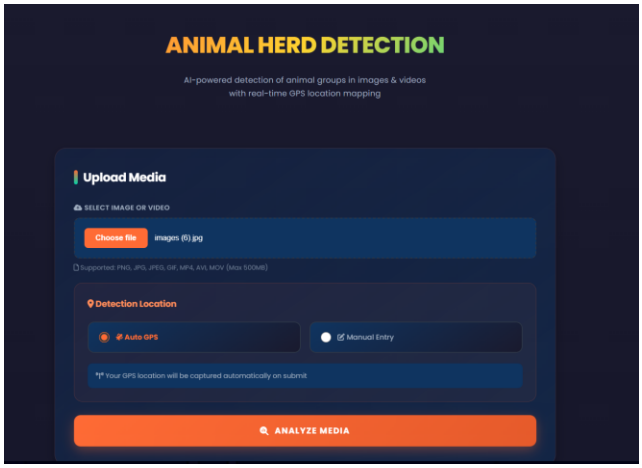
Model Loading Code:

```python
def load_yolo_model(self):
    weights_path = "models/yolov3.weights"
    config_path = "models/yolov3.cfg"
    names_path = "models/coco.names"

    self.net = cv2.dnn.readNet(weights_path, config_path)
    with open(names_path, "r") as f:
        self.classes = [line.strip() for line in f.readlines()]

    self.layer_names = self.net.getLayerNames()
    self.output_layers = [self.layer_names[i - 1] for i in self.net.getUnconnectedOutLayers()]
    self.model_loaded = True
    self.detection_method = "YOLO"
    print("✓ YOLO model loaded successfully")
```

2. Web Interface and File Upload

Users interact with the system through a Flask-based web interface featuring:

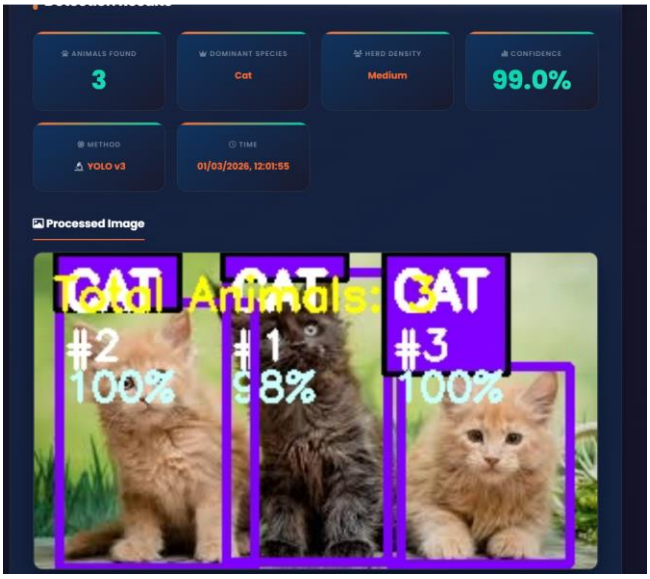
- Drag-and-drop file upload supporting images (PNG, JPG, JPEG, GIF) and videos (MP4, AVI, MOV)
- GPS coordinate input (manual entry or automatic detection)
- Real-time preview of uploaded media
- Progress indicators during processing
- Support for files up to 500MB



3. Image Preprocessing

Before detection, images undergo preprocessing to match YOLOv3 input requirements:

- Resize to 416×416 pixels (YOLO input size)
- Normalize pixel values to [0, 1] range by dividing by 255
- Convert from BGR to RGB color space
- Create 4D blob tensor with shape (1, 3, 416, 416)
- Swap RGB channels if needed for consistency



Preprocessing Code:

```
blob = cv2.dnn.blobFromImage(image, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
self.net.setInput(blob)
outs = self.net.forward(self.output_layers)
```

4. Animal Detection with YOLOv3

The YOLOv3 model processes the input image through its neural network:

- Forward pass through 106 convolutional layers
- Detections at three different scales (13×13, 26×26, 52×52 grids)
- For each grid cell: predicts 3 bounding boxes with 85 values each
- 85 values = 4 (box coordinates) + 1 (objectness score) + 80 (class probabilities)
- Filter detections with confidence > 50% threshold

5. Non-Maximum Suppression (NMS)

NMS eliminates redundant overlapping detections:

- Calculate Intersection over Union (IoU) between all detected boxes
- Keep boxes with IoU < 0.4 (40% overlap threshold)
- For overlapping boxes, retain only the one with highest confidence
- Result: Clean set of non-overlapping animal detections

6. Video Processing (Frame-by-Frame)

For video inputs, the system processes each frame individually:

- Open video file using OpenCV VideoCapture
- Extract frame rate (FPS) and resolution metadata
- Initialize VideoWriter for output with H.264 codec
- Read frames in sequence using a loop
- Apply YOLOv3 detection to each frame
- Draw bounding boxes and labels on detected animals
- Write annotated frame to output video
- Display processing progress (frame X of Y)
- Release resources and finalize output video file

7. Visualization and Annotation

Detection results are visualized with comprehensive annotations:

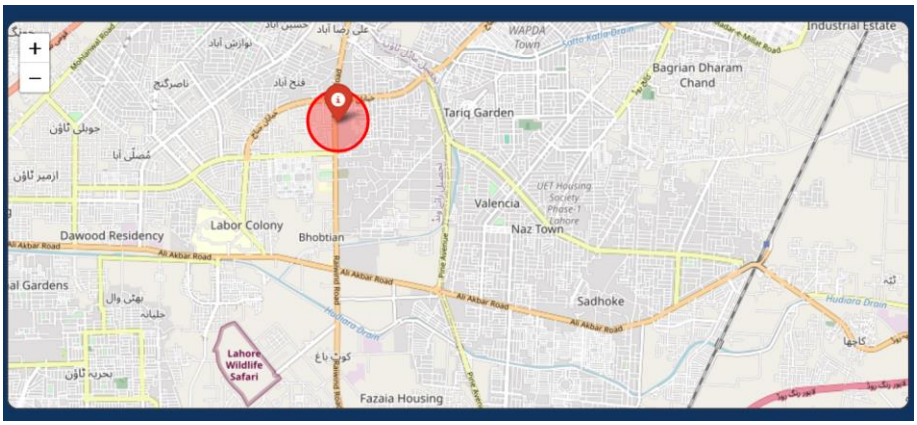
- Colored bounding boxes around each detected animal (species-specific colors)
- Species labels with confidence percentages
- Animal emoji icons for intuitive identification
- Count indicators for herd density
- Legend showing all detected species
- Timestamp and frame information for videos

8. Geospatial Mapping:

- Interactive OpenStreetMap base layer
- Red marker at detection coordinates
- Popup showing animal count and species distribution
- Circular detection zone (500-meter radius)
- Zoom controls and pan functionality
- Tooltip with location information

Map Generation Code:

```
def generate_map_with_location(latitude, longitude, animal_count, image_name):
    map_obj = folium.Map(
        location=[latitude, longitude],
        zoom_start=12,
        tiles='OpenStreetMap'
    )
```



9. Herd Density Classification

The system categorizes detection density into three levels:

Density Level	Animal Count
Low Density	1-5 animals
Medium Density	6-15 animals
High Density	16+ animals

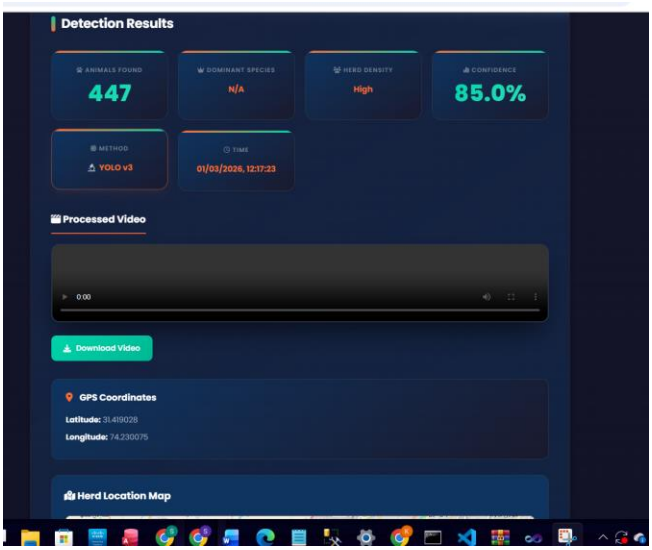
10. Results Delivery

The system returns comprehensive JSON response containing:

- Success status and error messages (if any)

- Detection method used (YOLO)
- Total animal count across all species
- Per-species breakdown with individual counts
- Herd density classification
- Confidence scores for each detection
- Annotated image/video file path
- Interactive map HTML
- Processing time statistics

Technologies Used

Technology	Purpose	Version
Python	Backend programming language	3.8+
Flask	Web framework for API and UI	≥2.3.0
OpenCV	Computer vision and image processing	≥4.8.0
YOLOv3	Deep learning object detection model	2018 Release
NumPy	Numerical computing and array operations	≥1.24.0
Folium	Interactive geospatial mapping	≥0.14.0
Werkzeug	WSGI web application library	≥2.3.0
Pillow	Image file handling	≥10.0.0
FFmpeg	Video encoding and conversion	Latest
HTML5/CSS3	Frontend web interface	Standard
JavaScript	Client-side interactivity	ES6+
COCO Dataset	Pre-trained model weights	2017 Release

Output: